

Building an AI Assistant with RAG, Embeddings & Vector Database

A complete technical journey from a simple LLM chatbot to a scalable RAG prototype.

RAG

AI

Laravel

FastAPI

ChromaDB

Embeddings

Project: AI Assistant

Core stack: React · Laravel · FastAPI · Sentence Transformers · ChromaDB · GPT-5.6-mini

Contents

1. Project Goal and Architecture
2. React Frontend
3. Laravel Backend
4. FastAPI AI Service
5. LLM Integration
6. Chat History
7. Why RAG?
8. Fake Embeddings and Cosine Similarity
9. Real Embeddings with Sentence Transformers
10. ChromaDB Vector Database
11. Retrieval, Top-K and Threshold
12. Prompt Builder and Grounded Answers
13. 10K Document Benchmark
14. Dataset Diversity and Benchmark Lessons
15. Scaling Roadmap: 100K → 1M
16. Final Architecture and Key Lessons

1. Project Goal and Architecture

The project was built from scratch to understand how modern AI applications are assembled rather than hiding the important steps behind a ready-made framework. The target was a knowledge-aware assistant that can retrieve project-specific information before asking an LLM to generate an answer.

High-level request flow

User
↓
React Frontend — Port 5173
↓
Laravel API — Port 8000
↓
FastAPI AI Service — Port 8001
↓
RAG Retrieval
↓
Relevant Context
↓
GPT-5.6-mini
↓
Final Answer

The separation gives the application clear responsibilities. Laravel remains the main application/backend layer, while Python owns AI-specific workloads such as embeddings, retrieval and LLM orchestration.

Layer	Technology	Responsibility
Frontend	React + Vite	Chat UI, user interaction, history display
Backend	Laravel 12	API gateway / application backend
AI Service	FastAPI + Python	RAG, embeddings, retrieval, LLM orchestration
Embedding	Sentence Transformers	Convert text into semantic vectors
Vector DB	ChromaDB	Store and search embeddings
LLM	GPT-5.6-mini	Generate the final grounded answer

2. React Frontend

The frontend was created with Vite and React. Axios was installed to communicate with the backend. The UI evolved into a simple chat interface that can keep and display conversation history.

```
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm install axios
npm run dev
```

Frontend responsibility

User enters a message
↓
React sends HTTP request to Laravel
↓
Laravel forwards the request to the AI service
↓
Response returns to React
↓
React renders the assistant message and history

The frontend is intentionally thin: it does not perform embeddings, vector search or direct database operations. Those concerns belong to the backend/AI service layers.

3. Laravel Backend

Laravel 12.69.2 was used as the application backend and API gateway. A [POST /chat](#) route validates the request and forwards it to FastAPI.

```
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Http;

Route::post('/chat', function (Request $request) {
    $request->validate([
        'message' => ['required', 'string'],
    ]);

    $response = Http::post('http://127.0.0.1:8001/chat', [
        'message' => $request->message,
    ]);

    return response()->json(
        $response->json(),
        $response->status()
    );
});
```

```
);  
});
```

This creates a clean boundary: React talks to Laravel, and Laravel talks to the AI service. Later, authentication, business rules, logging, rate limiting or persistence can be added to the Laravel layer without coupling those concerns to the Python code.

4. FastAPI AI Service

The AI service was isolated in a Python virtual environment. FastAPI provides the HTTP API, while Python packages handle the AI pipeline.

```
mkdir ai-service  
cd ai-service  
python3 -m venv .venv  
source .venv/bin/activate  
pip install fastapi uvicorn  
pip install openai python-dotenv  
pip install sentence-transformers  
pip install chromadb
```

The first service exposed `/health` and `/chat`. This gave a stable place to add AI capabilities incrementally instead of building the entire RAG system at once.

```
class ChatRequest(BaseModel):  
    message: str
```

The `/chat` endpoint eventually accepted both the current message and optional history.

```
class Message(BaseModel):  
    role: str  
    content: str  
  
class ChatRequest(BaseModel):  
    message: str  
    history: list[Message] = Field(default_factory=list)
```

5. LLM Integration

The first successful AI milestone was a direct call to the OpenAI Responses API using GPT-5.6-mini.

```
from openai import OpenAI

client = OpenAI()
response = client.responses.create(
    model="gpt-5.6-mini",
    input="Hello, how are you?"
)
print(response.output_text)
```

Initial chatbot

User question
↓
FastAPI
↓
OpenAI Responses API
↓
GPT-5.6-mini
↓
Generated response

This proved the basic LLM path worked. However, a general LLM response is not the same as a knowledge-grounded application. That gap led to the RAG part of the project.

6. Chat History

Conversation history was added so the model could receive previous messages along with the current question.

```
messages = []
for item in request.history:
    messages.append({
        "role": item.role,
        "content": item.content
    })
messages.append({
    "role": "user",
```

```
"content": request.message
}))
response = client.responses.create(
    model="gpt-5.6-mini",
    instructions="You are a helpful AI assistant. Explain technical concepts in simple terms",
    input=messages
)
```

An important distinction emerged: chat history represents the conversation, while RAG provides external/project knowledge. They can be combined, but they solve different problems.

Chat History → remembers what was discussed

↓

RAG → retrieves relevant knowledge

↓

LLM → uses both to formulate a response

7. Why RAG?

A normal chatbot follows the pattern Question → LLM → Answer. RAG inserts a retrieval stage before generation so the model receives relevant information from a controlled knowledge source.

Documents

↓

Chunks

↓

Embeddings

↓

Vector Database

↓

Similarity / Semantic Search

↓

Relevant Documents

↓

Context

↓

Prompt

↓

LLM



Grounded Answer

The project used a small **EcoLoop** dataset as the initial knowledge base (a fictional recycling-platform sample knowledge base):

```
documents = [  
    "EcoLoop is a recycling platform in Egypt.",  
    "EcoLoop allows users to sell recyclable materials.",  
    "EcoLoop supports materials such as plastic, paper, metals and used cooking oil.",  
    "Users can request a pickup from the platform.",  
    "The collected materials are sent for recycling.",  
]
```

8. Fake Embeddings and Cosine Similarity

Before using a real embedding model, a simple keyword-based vector was built to understand the mechanics. It was deliberately educational rather than production quality.

```
return [  
    1 if "ecoloop" in text else 0,  
    1 if any(word in text for word in [  
        "material", "materials", "plastic",  
        "paper", "metals", "metal",  
        "oil", "recycling"  
    ]) else 0,  
    1 if any(word in text for word in [  
        "pickup", "request", "collect"  
    ]) else 0,  
    1 if any(word in text for word in [  
        "weather", "sunny"  
    ]) else 0,  
]
```

The next step was Cosine Similarity. The idea is to compare two vectors and estimate how close their directions are. High similarity means the representations are close; low similarity means they are less related.

Text
↓
Simple vector representation
↓
Cosine similarity
↓
Rank documents
↓
Return top results

This stage was useful for understanding the algorithm, but it exposed the limitations of handcrafted features. Real semantic retrieval requires learned embeddings.

9. Real Embeddings with Sentence Transformers

The project moved from keyword vectors to a real embedding model: [all-MiniLM-L6-v2](#) through Sentence Transformers.

```
from sentence_transformers import SentenceTransformer

model = SentenceTransformer("all-MiniLM-L6-v2")

def embed_text(text):
    vector = model.encode(text)
    return vector.tolist()

def embed_texts(texts):
    vectors = model.encode(
        texts,
        batch_size=32,
        show_progress_bar=True
    )
    return vectors.tolist()
```

Human language text
↓
Sentence Transformer model
↓
Dense numerical vector



Vector stored in / searched from Vector DB

Batch embedding was introduced because embedding thousands of texts one at a time creates unnecessary overhead. Batch processing lets the model process multiple documents together.

10. ChromaDB Vector Database

Once the project moved beyond a few documents, an actual vector database was needed. ChromaDB was selected for the local prototype and installed with pip.

```
pip install chromadb

import chromadb

client = chromadb.PersistentClient(path="./chroma_db")
collection = client.get_or_create_collection(
    name="ecoloop_documents"
)
```

PersistentClient stores the vector database on disk rather than keeping everything only in process memory. Documents can be stored together with their embeddings and later queried semantically.

```
collection.add(
    ids=[str(index)],
    documents=[document],
    embeddings=[embedding],
)
```

11. Retrieval, Top-K and Threshold

Searching ChromaDB starts by embedding the user query and asking for the nearest documents.

```
def search_documents(query, top_k=2):
    query_embedding = get_embedding(query)
    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=top_k,
```

```
)  
return results
```

The project then introduced filtering so that an irrelevant nearest neighbor would not automatically become context for the LLM.

```
for document, distance in zip(  
    results["documents"][0],  
    results["distances"][0]  
):  
    if distance <= threshold:  
        documents.append(document)
```

A crucial discovery was that ChromaDB returns a distance metric in this configuration. Distance behaves opposite to similarity: lower is better. Therefore a comparison such as `distance <= threshold` is required.

```
Query embedding  
↓  
ChromaDB nearest neighbors  
↓  
Top-K candidates  
↓  
Distance check  
↓  
Relevant context OR reject
```

12. Prompt Builder and Grounded Answers

Retrieved documents need to be turned into context before they are sent to the LLM. A prompt builder was created to explicitly instruct the model to stay inside the provided context.

```
def build_context(results):  
    return "\n\n".join(  
        result["document"]  
        for result in results  
    )  
  
def build_prompt(question, context):  
    return f'''
```

Answer the question using only the provided context.

Rules:

- Do not use outside knowledge.
- Do not invent information.
- If the answer is not found in the context, say:
"I don't have enough information in the provided context."

Context:

{context}

Question:

{question}

Answer:

'''

User Question + Retrieved Context + Instructions / Rules

↓

Augmented Prompt

↓

LLM

↓

Grounded Answer

This design reduces the chance that the model answers an out-of-domain question using unrelated general knowledge, although prompt instructions alone are not a guarantee of perfect grounding.

13. 10K Document Benchmark

After the functional RAG prototype worked, the project shifted toward scalability testing. The first target was 10,000 documents.

```
DOCUMENT_COUNT = 10000
BATCH_SIZE = 500
documents = generate_documents(DOCUMENT_COUNT)

for start in range(0, DOCUMENT_COUNT, BATCH_SIZE):
    end = min(start + BATCH_SIZE, DOCUMENT_COUNT)
    batch = documents[start:end]
    embeddings = embed_texts(batch)
    ids = [str(i) for i in range(start, end)]
    collection.upsert(
        ids=ids,
```

```
documents=batch,  
embeddings=embeddings  
)
```

Upsert was used to make repeated benchmark runs safer: an existing ID can be updated instead of producing a duplicate-ID failure.

Observed local indexing measurements included approximately 10.61 seconds in one run and around 20–23 seconds in later runs. These are environment-specific measurements, not universal ChromaDB performance figures.

```
Generate 10,000 documents  
↓  
Split into batches of 500  
↓  
Batch embedding  
↓  
Upsert into ChromaDB  
↓  
Measure elapsed time  
↓  
Verify collection count
```

14. Dataset Diversity and Benchmark Lessons

The first 10K dataset exposed a major benchmarking problem: many records were effectively copies of the same EcoLoop text. A large number of duplicate documents can measure ingestion speed, but it does not provide a strong test of semantic retrieval quality.

A more varied generator was designed around topics such as materials, pickup, recycling, users, orders, companies, auctions, payments and platform operations.

```
topics = {  
    "materials": [...],  
    "pickup": [...],  
    "recycling": [...],  
    "users": [...],  
    "orders": [...],  
    "companies": [...],  
    "auctions": [...],  
    "payments": [...],
```

```
"platform": [...],  
}
```

The project also tested three useful query classes:

Query type	Example	What it tests
Relevant	What materials does EcoLoop accept?	Can the system retrieve known information?
Paraphrased relevant	How can users request a pickup?	Can semantic retrieval handle different wording?
Out-of-domain	What is Meta?	Can the system reject unrelated questions?

One of the most important debugging discoveries was that a relevant pickup document could be returned by ChromaDB with a distance around 1.86, but a threshold of 0.70 filtered it out. This demonstrated that retrieval can succeed while filtering fails.

```
Question: pickup  
↓  
Embedding  
↓  
ChromaDB finds the correct pickup document  
↓  
Distance ≈ 1.86  
↓  
Threshold = 0.70  
↓  
Result rejected  
↓  
Conclusion: filtering threshold needs evaluation
```

15. Scaling Roadmap: 100K → 1M

The next objective is to repeat the experiment at 100,000 and eventually 1,000,000 documents. The purpose is not simply to make the number bigger; it is to understand how indexing, storage and retrieval behave as the knowledge base grows.

10K — establish baseline
↓
100K — measure growth and bottlenecks
↓
1M — test large-scale local prototype

For each scale, the benchmark should record:

Metric	Question
Indexing time	How long does ingestion take?
Embedding time	How much time is spent creating vectors?
Storage size	How large is the ChromaDB data on disk?
Search latency	How many milliseconds does a query take?
Top-K retrieval	Does the correct document appear in top 1/3/5/10?
Rejection quality	Are unrelated queries rejected?

A strong future benchmark should also maintain a ground-truth set: each test query should have one or more known relevant documents. That makes retrieval quality measurable rather than subjective.

16. Final Architecture and Key Lessons

React — chat interface
↓
Laravel — application API / gateway
↓
FastAPI — AI orchestration
↓
Query embedding
↓
ChromaDB semantic retrieval
↓
Top-K + relevance filtering

↓
Context construction
↓
Prompt / grounding instructions
↓
GPT-5.6-mini
↓
Final response

The project has therefore evolved from a simple LLM call into a complete AI application prototype. Each layer was introduced only after the previous layer was understood and tested.

The most important lesson is that a good RAG answer is not created by the LLM alone. The final quality depends on the entire pipeline:

Dataset quality
↓
Chunking strategy
↓
Embedding model
↓
Vector database
↓
Retrieval strategy
↓
Top-K and threshold
↓
Context construction
↓
Prompt design
↓
LLM generation

Key principle: when an AI answer is wrong, debug the pipeline stage by stage instead of assuming the LLM is the problem. Retrieval can fail, filtering can fail, context can be incomplete, or the dataset can be poor.

The next practical milestone is to fix and verify the varied 10K dataset, rebuild the ChromaDB index from a clean state, establish a reliable retrieval benchmark, then scale to 100K and 1M documents.

